



---

# Successful Login Email Notification

Complete implementation guide — step by step, for  
Supabase + Next.js + Resend.

[Next.js 14.2.35](#)[Supabase Auth](#)[PostgreSQL](#)[Resend](#)[TypeScript](#)[Zod](#)

This guide shows you how to email a FitVerse user every time somebody signs in to their account, and how to keep your own team informed — without ever putting the login itself at risk. It is written for the FitVerse codebase specifically: it reuses the patterns already working in your waitlist route, your Supabase helpers and your Resend setup. Every technical term is explained the first time it appears.

# Contents

---

Sections 1–3 explain the idea and what you already have. Sections 4–14 are the build, in the order you should do it. Sections 15–16 are for when something breaks and for tracking your progress.

## UNDERSTANDING THE FEATURE

---

- 01 What Are We Building?** — the feature in plain language
- 02 The Recommended Approach** — the architecture, and why
- 03 Before You Start** — what FitVerse already has

## BUILDING IT

---

- 04 Step 1 — Supabase Login Detection**
- 05 Step 2 — Create the Database Table**
- 06 Step 3 — Create the Backend API Route**
- 07 Step 4 — Resend Configuration**
- 08 Step 5 — The User Login Email**
- 09 Step 6 — FitVerse Admin Notification**
- 10 Step 7 — Duplicate Protection**
- 11 Step 8 — Failure Handling**
- 12 Step 9 — Security & Privacy**
- 13 Step 10 — Testing**
- 14 Step 11 — Deployment**

## REFERENCE

---

- 15 Troubleshooting** — when things go wrong
- 16 Final Implementation Checklist**

### IMPORTANT — READ THIS FIRST

**The FitVerse landing page has no login yet.** The only authentication-adjacent thing in this repository is the waitlist form. The endpoint you are about to build lives here, but the actual sign-in happens inside the FitVerse mobile/web app.

This guide assumes **your app already uses Supabase Auth for email + password login**. If it does not yet, build that first — none of the steps below will do anything without it.

## Section 01

# What Are We Building?

A security feature. Whenever someone signs in to a FitVerse account, that account's owner gets an email telling them it happened. If it was them, they ignore it. If it wasn't, they now know within seconds that somebody else has their password.

## 1.1 What happens when a user logs in

Right now, when a FitVerse user types their email and password:

1. Supabase checks whether the password is correct.
2. If it is, Supabase creates a **session** — a record saying "this person is now signed in".
3. The app lets them in.
4. **Nothing else happens.** Nobody is told.

After you build this feature, one more thing happens, quietly, in the background:

5. Your system notices the new session, records it, and emails the user.

## 1.2 Why we need login notifications

Passwords leak. People reuse them across sites, sites get breached, and lists of email/password pairs get traded. When somebody uses a stolen FitVerse password, the login *looks completely normal* to your system — the password is correct, after all.

A login notification is the one thing that closes this gap. It goes to an address the attacker (usually) doesn't control, and it turns a silent break-in into something the real owner finds out about immediately.

### WHY THIS MATTERS MORE FOR FITVERSE THAN FOR MOST APPS

FitVerse stores **health data** — what people eat, how they sleep, their body metrics, and (with the For Women module) menstrual cycle information. This is among the most sensitive categories of personal data there is. A break-in here is not "someone saw my workout log" — it is a genuine privacy harm. Login alerts are a standard, expected protection for products holding this kind of data.

## 1.3 What the user receives

One short email: "Your FitVerse account was signed in to just now", with the time, the device, and a big obvious button — "**This wasn't me — reset my password**". Nothing technical, nothing alarming if it was them. Full design in Section 08.

## 1.4 What FitVerse receives

You originally wanted an email to your team for every single login. I am going to recommend something different — **one summary email per day** instead of one per login. The full reasoning is in Section 09, but the

short version is: an inbox that gets a thousand emails a day is an inbox nobody reads.

## 1.5 Why the backend must handle this

The obvious approach is to put the code in the app: "after login succeeds, send the email." **This is the wrong approach, and it is worth understanding why before you build anything.**

### Concept: client-side vs. server-side detection

#### WHAT "CLIENT-SIDE" MEANS

Code that runs on the user's own phone or browser — inside the FitVerse app itself.

#### WHAT "SERVER-SIDE" MEANS

Code that runs on machines you control — Supabase's servers and your Next.js backend. The user cannot see or change it.

#### WHY CLIENT-SIDE FAILS HERE

Two reasons. **First, it fires at the wrong times.** Supabase's client library has a listener called `onAuthStateChange` that emits a `SIGNED_IN` event. It sounds perfect, but it also fires when the app simply opens with a saved session, and sometimes when a browser tab regains focus. You would email people every time they opened the app. **Second, it can be skipped.** A modified app just doesn't make the call, and the alert silently never sends.

#### THE PRINCIPLE

**A security alert that can be skipped is not a security feature.** It must be triggered by something the attacker cannot switch off.

## 1.6 The whole system, in one picture

```
User types email + password in the FitVerse app
|
[1] Supabase Auth checks the password -> creates a NEW SESSION
|
[2] A database trigger notices "a new session row appeared"
    |      (sends an HTTP POST, with a secret header)
    v
[3] Your Next.js API route receives it
|
[4] Saves a row in login_events  <-- unique session_id blocks duplicates
|
[5] Resend emails the user "new sign-in detected"
|
[6] Row is updated: email sent / failed
|
[7] Separately, once a day, a digest email goes to your team
```

Notice that steps 2–7 happen *after* the user is already logged in. That ordering is deliberate, and it is what guarantees the rule we will come back to repeatedly: **a broken email system can never stop somebody signing in.**

## Section 02

## The Recommended Approach

You need Supabase, a trigger, a backend route, a database table, and Resend. These are not competing options — each does a different job. The only real decision is *how Supabase tells your backend that a login happened*, and Section 04 settles that.

### 2.1 The role of each component

| Component                                     | Its one job                                                           | What you do with it                                                                                                                   |
|-----------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Supabase Auth</b>                          | Checks the password and creates the session.                          | <b>Nothing.</b> You do not modify how login works at all. This is important — you are adding a listener, not changing authentication. |
| <b>Database trigger + <code>pg_net</code></b> | The doorbell. Notices a new session and calls your API.               | Write a small database function and attach it to the sessions table. Runs on Supabase's servers, so the app cannot skip it.           |
| <b>Next.js API route</b>                      | The brain. Verifies the call is genuine, records it, sends the email. | Create one new file, modelled on your existing waitlist route.                                                                        |
| <b>Supabase database</b>                      | Stores everything, and enforces the no-duplicates rule.               | Run one SQL migration.                                                                                                                |
| <b><code>login_events</code> table</b>        | The logbook. Also your duplicate-blocker and your retry list.         | Created by that migration. Protected by Row Level Security.                                                                           |
| <b>Resend</b>                                 | The postman. Actually delivers the email.                             | Already wired up in your project. You verify the domain and add a sender address.                                                     |
| <b>Admin digest</b>                           | One summary email per day to your team.                               | Build this <i>last</i> , after user emails work.                                                                                      |

### 2.2 Why this architecture suits FitVerse specifically

This is not a generic recommendation. It is shaped by what your project already contains:

- **You already have Resend working.** Your waitlist form sends a welcome email through it today. No new email provider, no new account, no new cost.
- **You already have a server-side Supabase admin client.** `lib/supabase.ts` exposes `getSupabaseAdmin()`, which can write to any table. Exactly what the new route needs.
- **You already have this exact route pattern.** `app/api/waitlist/route.ts` validates input, writes to Supabase, catches duplicate-key errors, hashes IP addresses and sends an email without letting failure break the main action. Your new route is that file with a different trigger.
- **You are a small team.** Keeping everything in Next.js means one language, one deployment, one place to look when something breaks.

**TIP – THE HONEST SUMMARY**

**You have already built this once.** The login notifier is your waitlist route with a different trigger and a different email template. When you get stuck, open the waitlist route and look at how it solved the same problem.

## 2.3 Why not a Supabase Edge Function?

Supabase offers **Edge Functions** — small programs that run on Supabase's own infrastructure. This would also work, and it is a perfectly respectable choice. I am not recommending it here because Edge Functions are written in Deno, deployed separately from your Next.js app, with their own secrets to manage and their own logs to check.

That means two stacks to maintain instead of one, for no functional gain. For a small pre-launch team, **one stack beats two**. If your team later grows and wants the notification logic to live closer to the database, moving it to an Edge Function is a straightforward migration.

**WARNING – DO NOT USE ONAUTHSTATECHANGE**

If you search online for "Supabase send email on login", most answers will tell you to listen to `onAuthStateChange` in your app. **Do not do this.** As explained in Section 1.5, it fires on app open and session restore, and it can be bypassed. Every problem it creates is one the server-side approach simply does not have.

## Section 03

## Before You Start

You are not starting from zero. Roughly 70% of the machinery already exists in the FitVerse codebase. This section shows you exactly what to reuse, so you don't rebuild things you already own.

### 3.1 Your reference implementation

#### READ THIS FILE FIRST

`app/api/waitlist/route.ts`

Read it start to finish before you write anything. Your new route is the same shape with a different trigger. Every pattern you need is already there, already working, already reviewed.

| Pattern in that file                                        | Lines   | Reuse it?                                    |
|-------------------------------------------------------------|---------|----------------------------------------------|
| <code>hashIP()</code> — SHA-256 + <code>IP_HASH_SALT</code> | 25–32   | <b>Yes, identically</b>                      |
| Zod schema + <code>safeParse</code> → 422                   | 35–47   | <b>Yes, different fields</b>                 |
| <code>getSupabaseAdmin()</code> for writes                  | 121     | <b>Yes</b>                                   |
| Duplicate detection via error <code>23505</code>            | 126–135 | <b>Yes — this is the core of the feature</b> |
| Try/catch → generic 500, details only to logs               | 157–163 | <b>Yes</b>                                   |
| <code>GET</code> returns 405                                | 166–168 | <b>Yes</b>                                   |
| CORS <code>secureHeaders()</code>                           | 50–66   | <b>No — deliberately skip (§6.2)</b>         |
| In-memory rate-limit <code>Map</code>                       | 8–22    | <b>No — don't copy (§6.2)</b>                |
| Fire-and-forget email <code>.catch()</code>                 | 145–153 | <b>Modify</b> — you'll await instead (§6.2)  |

### 3.2 Your existing configuration

| What                          | Where it lives                                                                     |
|-------------------------------|------------------------------------------------------------------------------------|
| <b>Resend client</b>          | <code>lib/resend.ts</code> — exports a ready-to-use <code>resend</code> object     |
| <b>Supabase server client</b> | <code>lib/supabase.ts</code> — <code>getSupabaseAdmin()</code> is the one you want |
| <b>Email templates</b>        | <code>fitverse/app/privacy/email-templates.ts</code>                               |
| <b>Environment variables</b>  | <code>.env.local</code> — not in the repo, it is gitignored                        |
| <b>Deployed site URL</b>      | <code>NEXT_PUBLIC_SITE_URL</code>                                                  |

### 3.3 Environment variables already in use

```

NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
RESEND_API_KEY
IP_HASH_SALT
NEXT_PUBLIC_SITE_URL

```

You will add exactly **two** new ones:

```

LOGIN_HOOK_SECRET      # a long random password shared with Supabase
ADMIN_ALERT_EMAIL      # where the daily team digest is sent

```

#### Concept: what `NEXT_PUBLIC_` means, and why it matters

##### WHAT IT IS

A prefix Next.js looks for on environment variable names.

##### WHAT IT DOES

Any variable starting with `NEXT_PUBLIC_` gets **baked into the JavaScript sent to the browser**. Anyone can open developer tools and read it. Variables *without* the prefix stay on the server and are never sent anywhere.

##### WHY YOU MUST CARE

`SUPABASE_SERVICE_ROLE_KEY` can read and write every row in your database, ignoring all security rules. If it ever got the `NEXT_PUBLIC_` prefix, you would be publishing full database access to every visitor.

##### THE GOOD NEWS

Your project already gets this right. `lib/supabase.ts` reads `SUPABASE_SERVICE_ROLE_KEY` with no prefix, and `lib/resend.ts` reads `RESEND_API_KEY` with no prefix. **Keep it that way.**

#### WARNING – FIX THIS BEFORE ADDING ANY NEW SECRET

Your `.gitignore` ignores `.env*.local` but **not** a plain `.env` file. If anyone on the team ever creates `.env`, your service role key and Resend key get committed to git — and once a secret is in git history, it is compromised forever, even if you delete it later.

**Add `.env` to `.gitignore` as step one.**

### 3.4 What you are reusing rather than rebuilding

| You could build...       | ...but you already have                                             |
|--------------------------|---------------------------------------------------------------------|
| An email sending service | Resend, configured and working in <code>lib/resend.ts</code>        |
| A database connection    | Lazy-loaded Supabase clients in <code>lib/supabase.ts</code>        |
| Input validation         | Zod, already a dependency, already used with <code>safeParse</code> |
| Duplicate handling       | The <code>23505</code> error-code check in the waitlist route       |



| You could build...          | ...but you already have                                                               |
|-----------------------------|---------------------------------------------------------------------------------------|
| IP anonymisation            | <code>hashIP()</code> — SHA-256 with a salt                                           |
| An email template structure | The HTML + plain-text pattern in <code>fitverse/app/privacy/email-templates.ts</code> |
| Safe error handling         | Generic responses to callers, real details only to <code>console.error</code>         |

## Section 04 · Step 1

# Supabase Login Detection

This is the one genuine decision in the whole project: how does Supabase tell your backend that a real login just happened? There are two candidates. I recommend one of them clearly, and explain the trade-off honestly.

## 4.1 Check whether the Password Verification Auth Hook exists

### Concept: an Auth Hook

#### WHAT IT IS

A place where Supabase lets you plug your own code into the login process itself.

#### WHY IT MIGHT HELP

The "Password Verification Attempt" hook fires every time Supabase checks a password, telling you *which user* and *whether the password was correct*. That is very close to what we want.

#### THE CATCH

Availability varies by Supabase plan and version, so you must check rather than assume.

### WHAT YOU NEED TO DO

1. Open your **Supabase Dashboard** and select your project.
2. In the left sidebar, click **Authentication**.
3. Look for a sub-item called **Hooks** (it may be labelled "Auth Hooks", and depending on your dashboard version it may live inside Authentication settings rather than as its own menu item).
4. If hooks are available, you will see a list of hook types — typically *Send Email*, *Send SMS*, *Custom Access Token*, *MFA Verification Attempt*, and *Password Verification Attempt*.
5. **Write down what you find.** You need this answer recorded even though, as explained next, you are probably not going to use it.

**How to tell it is unavailable:** the Hooks section is missing entirely, or the hooks are shown greyed out with an "upgrade your plan" prompt.

#### IMPORTANT – THE DASHBOARD CHANGES

Supabase redesigns its dashboard regularly, so exact menu labels may differ from what is written above. **Look for the concept, not the exact wording:** a section under Authentication that lists pluggable hooks. I am deliberately not inventing precise button names I cannot verify.

## 4.2 The decision — and my recommendation

### ✓ RECOMMENDED OPTION

**Use a database trigger on new sessions** — even if the Password Verification Hook turns out to be available in your project.

Here is the honest comparison:

| Capability                                    | Password Hook | New-session Trigger         |
|-----------------------------------------------|---------------|-----------------------------|
| Tells you a password specifically was checked | Yes           | No (covers all login types) |
| Gives you a <b>session ID</b>                 | No            | Yes                         |
| Gives you <b>device / browser</b> info        | No            | Yes                         |
| Gives you the <b>IP address</b>               | No            | Yes                         |
| Available on every Supabase plan              | Must verify   | Yes                         |

### WHY THE TRIGGER WINS

The password hook fires *before* a session exists. That means **no session ID** — and your entire duplicate-prevention design depends on that session ID being available. It also means no device information to put in the email. The trigger simply gives you more of what you need.

**And alerting on every login is better security, not worse.** If you add Google sign-in later, someone breaking into a Google account is exactly who you want to catch. Restricting alerts to password logins would be a downgrade.

### FALLBACK OPTION — FOR LATER, NOT NOW

If you later want to distinguish "signed in with a password" from "signed in with Google", the Password Verification Hook is where you would get that signal. Note whether it exists in your project, then set it aside. **Do not build on it today.**

## 4.3 Verify what your `auth.sessions` table contains

**Do this before writing anything else.** Supabase's internal tables vary between versions, so confirm rather than assume.

### WHAT YOU NEED TO DO

Dashboard → **SQL Editor** → **New query** → run this read-only query:

```
select column_name, data_type
from information_schema.columns
where table_schema = 'auth' and table_name = 'sessions'
order by ordinal_position;
```

You are looking for these columns:

- `id` — the session ID. **Essential.**
- `user_id` — who logged in. **Essential.**
- `created_at` — when. **Essential.**
- `user_agent` — the device/browser. *Nice to have.*
- `ip` — the IP address. *Nice to have.*

#### TIP

If `user_agent` and `ip` are missing, **everything still works** — you simply won't have device information in the email. Record what you find; the next step depends on it.

## 4.4 Enable the `pg_net` extension

### Concept: `pg_net`

#### WHAT IT IS

An add-on for PostgreSQL that lets the database itself make HTTP requests to the internet.

#### WHY WE NEED IT

Your database needs to phone your API route when a login happens. Databases normally cannot do this.

#### HOW IT HELPS SAFETY

`net.http_post` **queues the request and returns immediately.** It does not sit and wait for a reply, so it can never slow down or block a login.

### WHAT YOU NEED TO DO

Dashboard → **Database** → **Extensions** → search for `pg_net` → enable it.

## 4.5 Store your secret inside Supabase Vault

### Concept: the shared secret

#### WHAT IT IS

A long random password that only Supabase and your API route know.

#### WHY WE NEED IT

Your API endpoint sits on the public internet. Without a secret, *anyone* could call it and trigger fake "someone logged into your account!" emails to your users — a perfect phishing tool.

#### HOW IT WORKS

Supabase sends the secret in an HTTP header. Your route compares it to its own copy. No match, no action.

### WHAT YOU NEED TO DO

1. Generate a random string of **at least 32 characters** using a password manager. Do not type one yourself — human-chosen strings are guessable.

2. Go to Dashboard → **Project Settings** → **Vault** (it may also appear under Integrations).
3. Create a secret named `login_hook_secret` and paste the value in.
4. Keep the same value handy — it goes into your hosting environment as `LOGIN_HOOK_SECRET` in Section 07.

#### WHY USE VAULT INSTEAD OF TYPING IT IN THE FUNCTION?

If you paste the secret directly into the database function, anyone who can read your function definitions can read your secret. Vault keeps it encrypted and out of the function body.

## 4.6 Why the Webhooks dashboard UI probably won't work

You may be tempted by Dashboard → **Database** → **Webhooks**, which offers a friendly point-and-click way to watch a table. Try it, but be aware of a real limitation:

#### WARNING

The Webhooks UI usually only lists tables in the `public` schema. The table you need — `auth.sessions` — lives in the `auth` schema. **So you will most likely need to create the trigger by hand in the SQL Editor** rather than through the UI.

## 4.7 Creating the trigger — and the one rule that matters most

### Concept: a database trigger

#### WHAT IT IS

A rule that says "whenever X happens to this table, automatically run this code."

#### WHAT OURS DOES

"Whenever a new row is inserted into `auth.sessions`, send an HTTP POST to my API."

#### WHY IT IS RELIABLE

It lives inside the database. The app cannot skip it, disable it, or forget to call it.

#### WARNING - THE SINGLE MOST IMPORTANT DETAIL IN THIS GUIDE

**Wrap the entire trigger function body in an exception handler that swallows all errors.**

Triggers run *inside* the login transaction. If your function throws an error, the session insert is rolled back — and **the user cannot log in**. A single typo in your notification code would lock every FitVerse user out of their account.

An exception handler ending in `exception when others then return new;` guarantees that a broken notifier can never prevent a login. **Do not skip this.**

## What the function should do, in order

1. Try to read the secret from Vault.

FITVERSE - LOGIN NOTIFICATION IMPLEMENTATION GUIDE

SUPABASE + NEXT.JS + RESEND

2. Call `net.http_post` with your endpoint URL, a JSON body, and the headers below.

- Catch and swallow every possible error, always letting the login proceed.

### Trigger settings

| Setting     | Value                                                                                                                        |
|-------------|------------------------------------------------------------------------------------------------------------------------------|
| Timing      | <code>AFTER INSERT</code> — never <code>BEFORE</code> . "After" means the login has already succeeded before your code runs. |
| Table       | <code>auth.sessions</code>                                                                                                   |
| For each    | <code>ROW</code> — run once per new session, not once per statement.                                                         |
| HTTP method | <code>POST</code>                                                                                                            |
| URL         | <code>https://fitverse.app/api/auth/login-notify</code> — your <b>deployed</b> URL, not localhost.                           |
| Headers     | <code>Content-Type: application/json</code><br><code>x-fitverse-hook-secret: &lt;your secret&gt;</code>                      |

### The JSON body it should send

```
{
  "session_id": "<the new session's id>",
  "user_id":    "<the new session's user_id>",
  "occurred_at": "<created_at>",
  "user_agent": "<user_agent, or null>",
  "ip":         "<ip, or null>"
}
```

#### TIP - SEND THE RAW IP, STORE THE HASHED ONE

Send the **raw** IP address here. Your API route hashes it on arrival using the existing `hashIP()` function. That way the real IP travels over encrypted HTTPS but is **never written to disk**. Same approach your waitlist route already uses.

*Section 05 · Step 2*

## Create the Database Table

One table, `login_events`, does three jobs at once: it is your logbook, your duplicate-blocker, and your retry list. Run this SQL yourself in the Supabase SQL Editor.

### 5.1 The SQL migration

```
create table public.login_events (  
  id                uuid primary key default gen_random_uuid(),  
  session_id        uuid not null unique,  
  user_id            uuid not null references auth.users(id) on delete cascade,  
  occurred_at        timestampz not null,  
  
  user_agent         text,  
  ip_hash            text,  
  
  email_status        text not null default 'pending'  
                    check (email_status in ('pending', 'sent', 'failed', 'skipped')),  
  email_sent_at       timestampz,  
  email_error         text,  
  email_attempts      int not null default 0,  
  
  created_at         timestampz not null default now()  
);  
  
create index login_events_user_time_idx  
  on public.login_events (user_id, occurred_at desc);  
  
create index login_events_retry_idx  
  on public.login_events (email_status)  
  where email_status in ('pending', 'failed');  
  
create index login_events_digest_idx  
  on public.login_events (occurred_at desc);  
  
alter table public.login_events enable row level security;  
  
create policy "users read own login history"  
  on public.login_events  
  for select  
  using (auth.uid() = user_id);
```

## 5.2 Every column explained

### Necessary columns

| Column                    | What it is for                                                                                                                                                                                                                                     |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>           | The row's own identifier. Standard practice — every table gets one. <code>gen_random_uuid()</code> generates a random unique value automatically.                                                                                                  |
| <code>session_id</code>   | <b>The heart of the whole feature.</b> Supabase's ID for this specific login. Marked <code>unique</code> , which means the database <i>physically refuses</i> to store a second row with the same value. That refusal is your duplicate-blocker.   |
| <code>user_id</code>      | Who logged in. <code>references auth.users(id) on delete cascade</code> means if a user deletes their account, all their login history is deleted automatically — good for privacy compliance, and you get it for free.                            |
| <code>occurred_at</code>  | When the login actually happened, taken from Supabase.                                                                                                                                                                                             |
| <code>email_status</code> | Where the email got to: <code>pending</code> , <code>sent</code> , <code>failed</code> or <code>skipped</code> . The <code>check</code> constraint means a typo like <code>'sentt'</code> is rejected by the database rather than silently stored. |
| <code>created_at</code>   | When <i>you</i> recorded the row. Differs from <code>occurred_at</code> if a retry ran later — useful when debugging.                                                                                                                              |

### Useful columns

| Column                      | What it is for                                                                                                            |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>user_agent</code>     | The raw browser/device text string. You convert it into friendly wording like "Chrome on Windows" when writing the email. |
| <code>ip_hash</code>        | The scrambled IP address. Lets you notice "20 logins from one place" without ever storing anyone's actual address.        |
| <code>email_sent_at</code>  | The moment the email went out. Useful for spotting delays.                                                                |
| <code>email_error</code>    | The error message if sending failed. <b>Without this you will never know why an email didn't arrive.</b>                  |
| <code>email_attempts</code> | How many times you have tried. Lets your retry job give up after three attempts instead of looping forever.               |

### Columns I have deliberately left out

| Column                      | Why it is not there                                                                                                                                                                                                                                                      |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>email</code>          | You asked for it, and I would skip it. Storing the address <b>duplicates personal data</b> that can go out of date, and you can always fetch the current one from <code>auth.users</code> when you need it. One less copy of personal data is one less thing to protect. |
| <code>admin_notified</code> | Unnecessary once you use a digest. The digest simply asks "show me everything from the last 24 hours" using <code>occurred_at</code> . A per-row flag would be bookkeeping you never read.                                                                               |



| Column               | Why it is not there                                                                                                                                                                                             |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Approximate location | Needs a paid third-party IP-lookup service — an extra dependency and an extra cost. <b>Skip it in version 1.</b> Time plus device is enough for someone to recognise their own login. Add it later if you want. |

## 5.3 Keys, constraints and indexes explained

### Concept: primary key

#### WHAT IT IS

The column that uniquely identifies each row — here, `id`.

#### WHY WE NEED IT

So you can point at one specific row to update it (for example, to change `email_status` from `pending` to `sent`).

### Concept: unique constraint on `session_id`

#### WHAT IT IS

A rule telling the database "this value may appear at most once in this whole table."

#### WHY IT IS THE MOST IMPORTANT LINE IN THE MIGRATION

To prevent the same login email being sent twice, every login session gets a unique ID. The database allows that ID to appear only once. If Supabase accidentally sends the same event again, the database recognises the session already exists, rejects the insert, and your code simply skips sending another email.

#### WHY IT MUST LIVE IN THE DATABASE, NOT IN YOUR CODE

Serverless functions can run several copies of themselves at the same instant. Two copies could both ask "does this exist?", both get told "no", and both send an email. **The database cannot make that mistake** — a unique constraint is enforced at the storage layer, where only one write can win.

### Concept: indexes

#### WHAT THEY ARE

Like the index at the back of a book — a shortcut that lets the database find rows without reading every single one.

#### THE THREE WE CREATE

`login_events_user_time_idx` — makes "show me this user's recent logins" fast. You will want a "recent activity" screen eventually.

`login_events_retry_idx` — makes "find all emails that failed" fast. This is a *partial* index: it only covers `pending` and `failed` rows, so it stays small.

`login_events_digest_idx` — makes "show me everything from the last 24 hours" fast, for the admin digest.

## 5.4 Row Level Security — why nobody sees another user's history

### Concept: Row Level Security (RLS)

#### WHAT IT IS

A PostgreSQL feature where the database itself decides, row by row, who is allowed to see what.

#### WHY IT BEATS CHECKING IN YOUR CODE

If the rule lives in your application code, every new query is a new chance to forget it. If it lives in the database, **it cannot be forgotten** — the protection applies to every query from everywhere, automatically.

Three layers protect this table:

1. **enable row level security** — flips the default to *deny everything*. With RLS on and no matching policy, nobody reads anything at all.
2. **The select policy** — `auth.uid() = user_id` tells the database to compare the currently-logged-in user's ID against the row's owner. A user asking for someone else's rows gets an **empty result, not an error** — so there is nothing to probe at or guess from.
3. **No insert, update or delete policies** — deliberately absent. This means no ordinary user can ever write to this table, only read their own rows. Only your server writes.

#### IMPORTANT — HOW YOUR SERVER WRITES TO A PROTECTED TABLE

`getSupabaseAdmin()` in `lib/supabase.ts` uses the **service role key**, which bypasses RLS entirely. That is correct and intended — your backend needs to write rows for any user.

But it is *exactly why* that key must never reach a browser. RLS is your wall; the service role key is the master key that opens every door in it.

## Section 06 · Step 3

## Create the Backend API Route

| Detail         | Value                                                                                                  |
|----------------|--------------------------------------------------------------------------------------------------------|
| File location  | <code>app/api/auth/login-notify/route.ts</code>                                                        |
| Method         | <code>POST</code> only. Add a <code>GET</code> that returns 405, copying waitlist route lines 166–168. |
| Called by      | Your Supabase database trigger — never a browser.                                                      |
| Reference file | <code>app/api/waitlist/route.ts</code>                                                                 |

### 6.1 The flow at a glance

```

POST /api/auth/login-notify
|
|-- bad secret?      -> 401, stop
|-- bad JSON?       -> 400
|-- fails validation? -> 422
|
|-- insert row (status = pending)
|   |
|   +--- error 23505? -> 200, no email  <-- duplicate blocked
|
|-- fetch user email
|   |
|   +--- none?       -> mark skipped, 200
|
|-- await resend.send()
|-- update row (sent / failed)
|
+--- 200

```

### 6.2 Three deliberate differences from the waitlist route

You are copying that file's patterns, but three things must change. Understanding *why* each one changes will teach you more than the code itself.

#### 1 - NO CORS HEADERS

The waitlist route has `secureHeaders()` because a browser calls it, and browsers require CORS permission.

**This endpoint is called by your database, server to server.** Browsers should never touch it, so adding CORS would only widen your attack surface for no benefit. Leave it out.

## 2 - DO NOT COPY THE IN-MEMORY RATE-LIMIT MAP

The waitlist route keeps a counter in memory (line 8). On serverless hosting, each running instance gets its *own* memory, so the limit is unreliable — a fact worth knowing about that route too.

Here you do not need it. The **shared secret** stops strangers and the **unique session ID** stops repeats. Both are far stronger guarantees than a counter.

## 3 - AWAIT THE EMAIL INSTEAD OF FIRE-AND-FORGET

The waitlist route fires the email and forgets it (line 145) because a real person is staring at a loading spinner, waiting for a response.

**Here, nobody is waiting.** The login already finished several steps ago. So you can safely `await` the send and record whether it actually worked — which is what makes retries possible.

## 6.3 The logic, step by step

### STEP 1 – Check the secret, before anything else

Read the `x-fitverse-hook-secret` header and compare it to `process.env.LOGIN_HOOK_SECRET`. If missing or wrong → return **401** and stop immediately. Do nothing else — do not parse the body, do not log it.

#### Concept: timing-safe comparison

##### WHAT IT IS

A way of comparing two secrets that always takes exactly the same amount of time, regardless of how similar they are. In Node.js: `crypto.timingSafeEqual`.

##### WHY A NORMAL `===` IS UNSAFE

A normal comparison stops at the first wrong character. So comparing `"abc..."` takes microscopically longer than `"xbc..."` if your secret starts with "a". An attacker who measures thousands of requests can use those tiny differences to guess your secret one character at a time. This is called a **timing attack**.

##### WHAT YOU DO

Use `crypto.timingSafeEqual` instead of `===` for the secret check. Everywhere else, `===` is fine.

### STEP 2 – Parse the JSON

Wrap `request.json()` in a try/catch → return **400** on malformed JSON. Identical to waitlist route lines 100–107.

### STEP 3 – Validate with Zod

Build a schema requiring `session_id` (uuid), `user_id` (uuid) and `occurred_at` (datetime), with `user_agent` and `ip` optional/nullable. Use `safeParse` → return **422** on failure, exactly like waitlist route lines 110–116.

#### WHY VALIDATE AT ALL, IF ONLY SUPABASE CALLS THIS?

Because "only Supabase calls this" is an assumption, and security means not relying on assumptions. Validation also catches your own mistakes — a typo in the trigger's JSON body shows up as a clean 422 in your logs instead of a confusing database error later.

#### STEP 4 – Hash the IP

Call your existing `hashIP()` if an IP came through. Note the difference from the waitlist route: **there**, the IP is read from request headers because the user's browser makes the call. **Here**, the IP arrives in the request body, because the caller is your database, not the user.

#### STEP 5 – Insert the row first, before sending anything

Insert with `email_status: 'pending'`. Writing the record *before* attempting the email is what makes retries possible — if the send fails, you still have a record saying "this one needs another try".

#### STEP 6 – Catch error 23505

Check for it the same way waitlist route lines 127–131 does. If it fires, this session has already been handled — **return 200 immediately and send no email**.

This is the entire duplicate defence, and it is one `if` statement. For any *other* database error: log it and return **500**.

#### Concept: PostgreSQL error 23505

##### WHAT IT IS

The standard error code PostgreSQL returns for "unique constraint violated" — meaning "you tried to insert a value that already exists".

##### WHY YOU CHECK THE CODE, NOT THE MESSAGE

Error messages are written for humans and can change between database versions or languages. Error *codes* are stable and standardised.

##### WHERE YOU HAVE SEEN IT BEFORE

Your waitlist route already uses it to detect somebody signing up twice with the same email. Same code, same technique, new purpose.

#### STEP 7 – Look up the user's email address

The session data contains no email address, only a user ID. Use the admin client:

```
getSupabaseAdmin().auth.admin.getUserById(user_id)
```

This requires the service role key, which you already have configured. Pull out the email address and any display name stored in the user's metadata.

## STEP 8 – Handle a missing email address

If there is no address, update the row to `email_status: 'skipped'` and return **200**. This is *not* an error — there is simply nothing to send.

## STEP 9 – Send via Resend

Import the existing `resend` object from `lib/resend.ts`. Use the same call shape as waitlist route lines 146–152, but with:

```
from: "FitVerse Security <security@fitverse.app>"
```

and your new login template. Wrap the `await` in a try/catch.

## STEP 10 – Record the outcome

- **Success** → set `email_status: 'sent'`, `email_sent_at` to now, increment `email_attempts`.
- **Failure** → set `email_status: 'failed'`, store the message in `email_error`, increment `email_attempts`.

## STEP 11 – Return 200 either way

### IMPORTANT – THIS BIT IS COUNTERINTUITIVE

#### Even if the email failed, return 200.

A non-2xx response tells the caller "please retry me" — and `pg_net` will not retry anyway. So returning 500 achieves nothing except noise in your logs. The failure is *already recorded in the row*, which is exactly where your retry job will look for it.

Return 200 with a small body like `{ handled: true, email: "failed" }` so your logs stay readable and honest.

### Reserve non-200 responses for:

- **401** — bad or missing secret
- **400 / 422** — malformed or invalid request
- **500** — genuine server error, e.g. the database is unreachable

## Section 07 · Step 4

# Resend Configuration

## 7.1 Domain verification

### Concept: why a domain needs "verifying"

#### THE PROBLEM

Email was designed without security. Anyone can technically send an email claiming to be `security@fitverse.app`.

#### THE SOLUTION

You add records to your domain's DNS settings that prove to receiving mail servers "yes, Resend really is allowed to send on my behalf."

#### WHAT HAPPENS IF YOU SKIP IT

Gmail and Outlook put your mail in spam, or reject it outright.

### WHAT YOU NEED TO DO

Resend Dashboard → **Domains** → find your `fitverse.app` entry. **Confirm it shows "Verified"** before relying on it.

#### TIP – IT MAY ALREADY BE DONE

Your code already sends from `hello@fitverse.app` (waitlist route, line 147), so the domain is probably already verified. Check rather than assume.

## 7.2 The three DNS records

| Record       | Who provides it                    | What it does, in plain terms                                                                                                  |
|--------------|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>SPF</b>   | Resend gives you the exact record  | A public list of who is allowed to send email as <code>fitverse.app</code> . Like a guest list on the door.                   |
| <b>DKIM</b>  | Resend gives you the exact record  | A cryptographic signature on every email, so receivers can prove it really came from you and wasn't altered. Like a wax seal. |
| <b>DMARC</b> | <b>You write this one yourself</b> | Tells receiving servers what to do with mail that fails the two checks above. Like instructions to the doorman.               |

You add all three at your domain registrar (wherever you bought `fitverse.app`).

### Setting up DMARC safely

Add a TXT record at `_dmarc.fitverse.app`.

**WARNING – DO NOT START STRICT**

Start with a monitoring-only policy ( `p=none` ) plus a reporting address, and watch for about a week. Then tighten to `quarantine` , and eventually `reject` .

Going straight to `reject` can **silently kill your own email** if anything is misconfigured — and you won't find out, because the messages simply vanish.

**WHY DELIVERABILITY MATTERS MORE HERE THAN FOR THE WAITLIST**

A welcome email landing in spam is a minor annoyance. **A security alert landing in spam is a security feature that does not exist.** This is the one email you cannot afford to have filtered.

## 7.3 Sender address

Use `security@fitverse.app` with the display name **"FitVerse Security"**.

**WHY NOT REUSE HELLO@FITVERSE.APP?**

Keep it separate from your existing `hello@fitverse.app` . If someone marks a waitlist marketing email as spam, you do not want that damaging the reputation of your security alerts. Different purposes, different addresses, separate reputations.

## 7.4 Where the API key belongs — and where it must not

|                         | Location                                                                                                                                                                                   |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MUST be in</b>       | <code>.env.local</code> for local development · Vercel Environment Variables for production · read only through <code>process.env.RESEND_API_KEY</code> inside <code>lib/resend.ts</code>  |
| <b>MUST NEVER be in</b> | Any <code>NEXT_PUBLIC_*</code> variable · any file marked <code>"use client"</code> · any file committed to git · any error message returned to a caller · any screenshot or Slack message |

## 7.5 The two new environment variables

| Name                           | Value and notes                                                                               |
|--------------------------------|-----------------------------------------------------------------------------------------------|
| <code>LOGIN_HOOK_SECRET</code> | 32+ random characters. <b>Must exactly match</b> the Vault secret you created in Section 4.5. |
| <code>ADMIN_ALERT_EMAIL</code> | Where the daily digest goes. Use a team alias, not a personal inbox — people leave companies. |



## 7.6 Test Resend on its own, before connecting anything

### TIP – YOU HAVE A SHORTCUT MOST PROJECTS DON'T

**Submit your own waitlist form.** It already runs the complete path: a Supabase insert followed by a Resend send, using the same client and the same API key.

If a welcome email arrives → Resend works and your domain is verified. If it does not → fix that *first*, in isolation, before you add login notifications on top. Debugging one broken thing is easy; debugging two at once is miserable.

### IMPORTANT – A SILENT-FAILURE TRAP IN YOUR CURRENT SETUP

`lib/resend.ts` falls back to the placeholder string `"re_placeholder"` when the API key is missing. This means a misconfigured deployment does **not** crash — it just quietly fails to send anything.

That is acceptable for waitlist mail. For security alerts you may prefer it to fail loudly at startup so you find out immediately. Your call — but decide consciously rather than by accident.

Section 08 · Step 5

# The User Login Email

## 8.1 Subject line

Recommended: "New sign-in to your FitVerse account"

WHY NOT "FITVERSE – NEW LOGIN DETECTED"?

Two reasons. **All-caps brand names raise spam scores** with mail filters. And **"Detected" sounds like something went wrong** even when it was the user themselves — which trains people to feel anxious about a routine message, or to start ignoring it.

## 8.2 Recommended email structure

From: FitVerse Security <security@fitverse.app>  
Subject: New sign-in to your FitVerse account

Hi [Name],

Your FitVerse account was signed in to just now.

When: 18 August 2026, 9:42 PM IST  
Device: Chrome on Windows  
Account: v\*\*\*\*l@gmail.com

If this was you, no action needed.

If this wasn't you, secure your account now:

[ Reset your password ] -> <https://fitverse.app/reset-password>

Questions? Reply to this email or contact hello@fitverse.app.

– The FitVerse team

## 8.3 What to include, and why each part earns its place

| Item                         | Why it is there                                                                                                            |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Their name                   | Generic, nameless emails read as phishing. Personalisation is a trust signal.                                              |
| Login time, with timezone    | Your users are in India — say <b>IST</b> . A bare "3:12 PM" with no timezone is useless for judging "was that me?"         |
| Device / browser             | The fastest way for someone to recognise "that definitely wasn't me — I don't own a Mac."                                  |
| Account email, partly masked | Confirms <i>which</i> account this is about, while masking limits the damage if the email gets forwarded or screenshotted. |

| Item                            | Why it is there                                                                                  |
|---------------------------------|--------------------------------------------------------------------------------------------------|
| <b>Approximate location</b>     | <i>Optional, skip in v1.</i> Requires a paid IP-lookup service. Useful later, not essential now. |
| <b>A clear security warning</b> | Tells them plainly what to do if it wasn't them.                                                 |
| <b>One obvious action</b>       | A password reset link. <b>An alert with nothing to do is just noise.</b>                         |

## 8.4 Never include

### WARNING – MUST NEVER APPEAR IN THIS EMAIL

- **The password.** Ever, in any form, even partially.
- **JWTs, access tokens, refresh tokens, session tokens.**
- **API keys or secrets** of any kind.
- **A link that logs somebody in automatically.** The reset link must require identity confirmation — otherwise your security email becomes a skeleton key sitting in an inbox.
- **The full session ID.** Internal plumbing, of no use to the reader.
- **The raw IP address.** You only stored a hash anyway, and it means nothing to most users.
- **Any other user's information.**

### TIP – THE ONE-QUESTION TEST

Before sending any version of this email, ask: **"If this were forwarded to a complete stranger, could they do any harm with it?"** The answer must be *no*. If it is anything else, remove something.

## 8.5 Where to put the template

Your existing templates live in `fitverse/app/privacy/email-templates.ts`, and each has an **HTML version and a plain-text version**. Follow that two-version pattern — some email clients block HTML, and a plain-text fallback means your security alert still arrives readable.

### IMPORTANT – A NOTE ABOUT THAT FILE PATH

That path is odd: an email template sitting inside a folder named `privacy`, inside a duplicated `app` tree. It works, so you are not obliged to move it — but you may prefer to put the new template somewhere clearer, such as `lib/emails/`.

## Section 09 · Step 6

# FitVerse Admin Notification

You originally asked for an email to your team on every login. I am going to recommend against it — and explain the reasoning clearly enough that you can disagree if you want to.

## 9.1 The three options compared

|          | Option                        | Assessment                                                                                                                                                                                                                                                                                                                                |
|----------|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A</b> | Email for <b>every</b> login  | Works today with zero users. At 500 users logging in twice a day, that is <b>1,000 emails a day</b> to your team. You will create an inbox filter within a week — and then the one login that actually mattered gets filtered too. <b>Option A does not fail loudly; it fails by being ignored</b> , which is worse than not building it. |
| <b>B</b> | <b>Daily digest</b>           | One readable email per day. Uses the same table, no new plumbing. Gives you the data you will need to define "suspicious" later.                                                                                                                                                                                                          |
| <b>C</b> | <b>Suspicious logins only</b> | Genuinely the best long-term answer — but it needs a definition of "suspicious", which needs device fingerprinting and a baseline of normal behaviour. <b>You have no users yet, so you have no baseline</b> . Not buildable today.                                                                                                       |

### ✓ RECOMMENDATION FOR FITVERSE RIGHT NOW

#### Option B — the daily digest.

- One email a day, so it stays readable and you will actually read it.
- Uses the same `login_events` table — no new infrastructure.
- Collects exactly the data you need to *define* "suspicious" later, so it is the natural stepping stone to Option C in six months.
- During early testing you can run it hourly, or point it at Slack, without changing the design at all.

## 9.2 How you would build it — later

### IMPORTANT – SEQUENCING

**Build this after the user emails are fully working.** Do not do both at once. If something breaks, you want to know which half broke.

1. **A second endpoint** — `app/api/admin/login-digest/route.ts`, protected by the same shared-secret pattern as your login route.
2. **It queries** `login_events` for rows where `occurred_at` falls in the last 24 hours, and counts: total logins, unique users, and any rows where `email_status = 'failed'`.
3. **It sends one email** to `ADMIN_ALERT_EMAIL` via Resend.

4. **Something calls it once a day** — either Vercel Cron (configured in `vercel.json`) or Supabase's `pg_cron` extension. **Vercel Cron is simpler**, since the endpoint already lives there.

**TIP**

The `login_events_digest_idx` index from Section 05 exists specifically to make this query fast. You already built the foundation for this step.

## Section 10 · Step 7

# Duplicate Protection

## 10.1 The idea, in everyday terms

### THE CLOAKROOM TICKET

Think of a **cloakroom ticket**. You hand over your coat and get ticket #4471. If you turn up with #4471 a second time, the attendant says "already collected" — they do not hand you a second coat. The ticket number is unique, so it can only be redeemed once.

**session\_id** is the ticket. The unique constraint is the attendant. Even if your API is called five times with the same session ID, only the first insert succeeds — the other four hit error **23505** and get turned away.

### IMPORTANT — WHY THE CHECK MUST LIVE IN THE DATABASE

Serverless functions can run several copies of themselves at the same instant. Two copies could both check "does this exist?", both be told "no", and both send an email.

**The database cannot make that mistake.** A unique constraint is enforced at the storage layer, where only one write can possibly win. This is why we do not write duplicate-checking logic in TypeScript.

## 10.2 Every scenario, and exactly what happens

| What happens                             | New session?      | Email? | Why                                                                                                                                                                        |
|------------------------------------------|-------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User types email + password and signs in | Yes               | YES    | This is the real thing. Exactly what you want.                                                                                                                             |
| Supabase sends the webhook twice         | No — same session | No     | Second insert hits <b>23505</b> ; the route returns 200 and stops.                                                                                                         |
| User refreshes the page                  | No                | No     | The existing session is reused. Nothing new is created.                                                                                                                    |
| App is closed and reopened               | No                | No     | The saved session is restored, not recreated.                                                                                                                              |
| Token refreshes (roughly hourly)         | No                | No     | <b>The key one.</b> A refresh updates the existing session's timestamp — it does not insert a new row. Your trigger only fires on <b>INSERT</b> , so it never runs at all. |
| Session restored from storage            | No                | No     | Nothing is inserted, so nothing fires.                                                                                                                                     |
| User logs out, then logs back in         | Yes, brand new    | YES    | New session, new ID, new email. Correct — this genuinely is a new login.                                                                                                   |
| Same person on phone and laptop          | Two sessions      | TWO    | Correct — two devices are two things worth knowing about.                                                                                                                  |

**TIP – THE INSIGHT WORTH REMEMBERING**

The scenario you were most worried about — the hourly token refresh — is handled **by physics rather than by logic**. There is no `INSERT`, so there is no trigger, so there is no email. Nothing to configure, nothing to get wrong.

**This is precisely why the server-side design beats client-side detection.** You are not filtering out false alarms — they never occur in the first place.

## Section 11 · Step 8

# Failure Handling

## THE GOVERNING RULE OF THIS ENTIRE FEATURE

**A failed email must NEVER prevent a user from logging into FitVerse.**

Your architecture enforces this structurally — the notification runs *after* the login, in a separate process. Even total failure of every component below leaves login completely untouched.

## 11.1 Every failure scenario

| # | Situation                               | What happens to login       | What happens to the email                                                                                                                                                                               |
|---|-----------------------------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | API route is down                       | Unaffected — works normally | <code>net.http_post</code> queues the call, it fails, the database moves on. No email, no record.                                                                                                       |
| 2 | Resend is down                          | Unaffected — works normally | Row is already saved as <code>pending</code> . The send throws, gets caught, row is marked <code>failed</code> with the error text. The retry job picks it up later. <b>Nothing is lost.</b>            |
| 3 | Database insert fails (not a duplicate) | Unaffected — works normally | Logged, route returns 500. No email, no record. Rare, and visible in monitoring.                                                                                                                        |
| 4 | Email sending fails                     | Unaffected — works normally | Same as #2 — recorded as <code>failed</code> , not lost.                                                                                                                                                |
| 5 | Webhook fires twice                     | Unaffected — works normally | Second insert hits <code>23505</code> → return 200, no second email.                                                                                                                                    |
| 6 | User has no email address               | Unaffected — works normally | Row marked <code>skipped</code> , return 200. Not treated as an error.                                                                                                                                  |
| 7 | Invalid request body                    | Unaffected — works normally | Zod rejects it → 400 or 422. Nothing saved, nothing sent.                                                                                                                                               |
| 8 | Someone calls the endpoint manually     | Unaffected — works normally | No valid secret → <b>401 immediately</b> , before any parsing. Nothing saved, no email. <b>This is the attack you most need to block</b> — otherwise anyone could spam a user's inbox with fake alerts. |
| 9 | Same session ID submitted twice         | Unaffected — works normally | Same as #5. The database refuses it.                                                                                                                                                                    |

## TIP — NOTICE THE PATTERN

Every single row says login is unaffected. **That is not luck** — it is what "trigger fires after the login, in a separate process, wrapped in an exception handler" buys you. The safety is designed in, not bolted on.



## 11.2 The retry job

**Build this after the main flow works.**

A scheduled task that finds rows where `email_status` is `'pending'` or `'failed'` and `email_attempts < 3`, then tries sending again. The `login_events_retry_idx` index exists for exactly this query.

### IMPORTANT – DO NOT RETRY FOREVER

Cap it at three attempts. After that, leave the row marked `failed` and let the daily digest surface it. Infinite retries turn one broken email into thousands of pointless API calls, and can get your Resend account rate-limited.

## Section 12 · Step 9

## Security & Privacy

Each item below includes *why* it matters. Security checklists you follow without understanding are the ones you quietly skip when you are in a hurry.

### 12.1 Secrets

- ☐ `LOGIN_HOOK_SECRET` is 32+ random characters from a password manager — not something you typed.  
*Why: human-invented strings are guessable; generated ones are not.*
- ☐ The same value exists in Supabase Vault **and** your hosting environment — and nowhere else.  
*Why: every extra copy is another place it can leak from.*
- ☐ Compared with `crypto.timingSafeEqual`, never `===`.  
*Why: prevents timing attacks that reveal your secret one character at a time.*
- ☐ The trigger function reads it from Vault, not hardcoded in the function body.  
*Why: anyone who can read your database functions would otherwise read your secret.*
- ☐ `SUPABASE_SERVICE_ROLE_KEY` has **no** `NEXT_PUBLIC_` prefix — already correct in `lib/supabase.ts`.  
*Why: this key bypasses all security rules. In a browser it is total database compromise.*
- ☐ `RESEND_API_KEY` likewise — already correct in `lib/resend.ts`.  
*Why: a leaked key lets anyone send email as FitVerse.*
- ☐ `.env` added to `.gitignore` — currently only `.env*.local` is covered.  
*Why: a secret committed to git is compromised permanently, even after deletion.*
- ☐ No secret ever appears in a response body or an error message.  
*Why: error responses get logged, screenshoted and pasted into chat.*

### 12.2 The endpoint

- ☐ Secret is checked **first**, before parsing anything at all.  
*Why: gives an attacker the smallest possible surface to interact with.*
- ☐ `POST` only; `GET` returns 405.  
*Why: reduces accidental exposure and search-engine crawling.*
- ☐ No CORS headers — this is server-to-server.  
*Why: CORS would invite browser traffic you never want.*
- ☐ Every input validated by Zod before it touches the database.  
*Why: malformed data becomes a clean 422 instead of a confusing database error.*
- ☐ Error responses are generic; real details go only to `console.error`, matching waitlist route line 137.  
*Why: detailed errors tell attackers how your system is built.*

### 12.3 Data protection

- ☐ RLS enabled on `login_events`, with only a read-your-own policy.  
*Why: makes leaking another user's login history structurally impossible, not just unlikely.*

FITVERSE - LOGIN NOTIFICATION IMPLEMENTATION GUIDE

SUPABASE + NEXT.JS + RESEND

- ☐ IPs hashed with your existing `hashIP()` — raw IPs never stored.  
*Why: you keep the ability to spot patterns without holding data that can locate someone.*
- ☐ `IP_HASH_SALT` is set to a real value in production.  
*Why: waitlist route line 27 falls back to the literal string `"fitverse-salt"`, which is visible in your source code and therefore public. **A known salt makes hashes reversible by brute force.** Check this one.*
- ☐ A retention policy — delete rows older than 90 days on a schedule.  
*Why: data you no longer hold cannot be breached, subpoenaed or leaked.*
- ☐ `on delete cascade` means account deletion removes login history automatically.  
*Why: satisfies "right to erasure" without anyone remembering to do it manually.*

## 12.4 Rate limiting

- ☐ Do not copy the in-memory `Map` from the waitlist route.  
*Why: on serverless, each instance has separate memory, so the limit does not reliably hold.*
- ☐ The unique constraint plus the shared secret are your real protections.  
*Why: both are enforced in places an attacker cannot reach.*
- ☐ Optionally add a per-user cooldown query.  
*Why: so somebody logging in 20 times an hour does not receive 20 emails and learn to ignore them.*

## 12.5 Logging

- ☐ Never log full request bodies containing personal data.  
*Why: logs are widely readable, retained a long time, and often shipped to third-party services.*
- ☐ Log outcomes and error codes, not payloads.  
*Why: gives you everything needed to debug, without creating a second copy of personal data.*

## 12.6 Privacy and compliance

### WARNING – A COMPLIANCE STEP YOU MUST NOT SKIP

**Update** `app/privacy/page.tsx`. Its "Information We Collect" list does not currently mention login metadata. Under **India's DPDP Act**, you should be able to state what you collect, why you collect it, and how long you keep it.

- ☐ Add login metadata (time, device, hashed IP) to the Privacy Policy's collection list.  
*Why: legally required, and it is the honest thing to do.*
- ☐ State the 90-day retention period explicitly.  
*Why: "how long" is a specific requirement under both DPDP and GDPR.*
- ☐ Say that security alerts are transactional and have no unsubscribe link.  
*Why: this is normal and legal for security messages — but you must say so, rather than leaving people confused about why they cannot opt out.*
- ☐ Mask the email address inside the email itself.  
*Why: limits exposure if the message is forwarded or shown on a shared screen.*

## Section 13 · Step 10

## Testing

Test in this order. Do not move to the next stage until the current one fully passes — debugging one broken thing is easy, debugging four at once is not.

### 13.1 Stage 1 — Before connecting anything

| Test                | What you do                                 | What should happen     | Email?     | Database result              |
|---------------------|---------------------------------------------|------------------------|------------|------------------------------|
| <b>Resend works</b> | Submit your own waitlist form               | Welcome email arrives  | <b>Yes</b> | Row in <code>waitlist</code> |
| <b>Table exists</b> | Run <code>select * from login_events</code> | Empty result, no error | —          | No rows yet                  |
| <b>RLS blocks</b>   | Query as a normal logged-in user            | Zero rows returned     | —          | Nothing visible              |

### 13.2 Stage 2 — The endpoint alone (curl or Postman, no Supabase)

| Test                     | What you do                                    | What should happen   | Email?                 | Database result                    |
|--------------------------|------------------------------------------------|----------------------|------------------------|------------------------------------|
| <b>Unauthorized</b>      | POST with no secret header                     | <b>401</b>           | <b>No</b>              | <b>No row</b>                      |
| <b>Wrong secret</b>      | POST with an incorrect secret                  | <b>401</b>           | <b>No</b>              | <b>No row</b>                      |
| <b>Invalid body</b>      | Valid secret, <code>{"hello":"world"}</code>   | <b>422</b>           | <b>No</b>              | <b>No row</b>                      |
| <b>Bad JSON</b>          | Valid secret, malformed body                   | <b>400</b>           | <b>No</b>              | <b>No row</b>                      |
| <b>Wrong method</b>      | Send a GET request                             | <b>405</b>           | <b>No</b>              | <b>No row</b>                      |
| <b>Valid call</b>        | Valid secret, real user ID, made-up session ID | <b>200</b>           | <b>Yes</b>             | <b>One row</b> , <code>sent</code> |
| <b>Duplicate session</b> | Repeat the exact same call                     | <b>200</b>           | <b>No second email</b> | Still exactly one row              |
| <b>Unknown user</b>      | Valid body, random user ID                     | 200 or 500, no crash | <b>No</b>              | Check behaviour matches your code  |

#### IMPORTANT — THE TEST THAT MATTERS MOST IN THIS STAGE

**The duplicate test.** Run the same call three times and confirm you get exactly **one** row and **one** email. If you get two of either, stop and fix it before going further — everything downstream depends on this working.

### 13.3 Stage 3 — The full flow, with the trigger connected

| Test                    | What you do                            | What should happen       | Email?          | Database result                                      |
|-------------------------|----------------------------------------|--------------------------|-----------------|------------------------------------------------------|
| <b>Successful login</b> | Sign in with the correct password      | Alert arrives in seconds | <b>Yes</b>      | <b>New row, <code>sent</code></b>                    |
| <b>Failed login</b>     | Sign in with the <b>wrong</b> password | Nothing at all happens   | <b>No</b>       | <b>No row — no session was created</b>               |
| <b>App restart</b>      | Close and reopen the app               | Silence                  | <b>No</b>       | <b>No new row</b>                                    |
| <b>Session restore</b>  | Return with a saved session            | Silence                  | <b>No</b>       | <b>No new row</b>                                    |
| <b>Token refresh</b>    | Stay logged in ~1 hour past expiry     | Silence                  | <b>No</b>       | <b>No new row</b>                                    |
| <b>Logout → login</b>   | Log out, then log back in              | New alert arrives        | <b>Yes</b>      | <b>Second row, different <code>session_id</code></b> |
| <b>Two devices</b>      | Log in on phone and laptop             | Two alerts               | <b>Two</b>      | <b>Two rows</b>                                      |
| <b>Multiple users</b>   | Two different accounts log in          | Each gets their own only | <b>One each</b> | <b>Correct <code>user_id</code> on each row</b>      |

#### TIP — BE PATIENT WITH THE TOKEN-REFRESH TEST

Waiting an hour is boring, but this is the exact failure mode that makes login notifications annoying enough that users disable them. **You want to see it not happen with your own eyes.**

### 13.4 Stage 4 — Failure simulation

| Test                    | How to simulate it                                                | What should happen                                     | Login still works? |
|-------------------------|-------------------------------------------------------------------|--------------------------------------------------------|--------------------|
| <b>Resend down</b>      | Temporarily set an invalid <code>RESEND_API_KEY</code>            | Row saved as <code>failed</code> with an error message | <b>YES</b>         |
| <b>API route down</b>   | Point the trigger at a wrong URL                                  | No row, no email, nothing breaks                       | <b>YES</b>         |
| <b>Database rejects</b> | Manually insert a duplicate <code>session_id</code> in SQL Editor | PostgreSQL raises <code>23505</code>                   | <b>YES</b>         |

#### WARNING — THE SINGLE MOST IMPORTANT TEST IN THIS GUIDE

##### Break Resend completely, then log in.

If you get into FitVerse normally, your architecture is correct. If login hangs or fails, **your trigger's exception handler is not working** — fix it before going anywhere near production. This is the difference between a nice feature and locking every user out of their account.

## Section 14 · Step 11

# Deployment

```

LOCAL SETUP
|
SUPABASE SETUP
|
RESEND SETUP
|
ENVIRONMENT VARIABLES
|
DATABASE MIGRATION
|
API DEPLOYMENT
|
WEBHOOK / HOOK CONFIGURATION
|
TESTING
|
PRODUCTION

```

## 14.1 Locally

- ☐ Add `LOGIN_HOOK_SECRET` and `ADMIN_ALERT_EMAIL` to `.env.local`
- ☐ Confirm `.env` is in `.gitignore` before anything else
- ☐ Build the route and test it with curl, using the Stage 2 table from Section 13

### IMPORTANT – A LIMITATION TO PLAN AROUND

**Supabase cannot call `http://localhost`**. Your database lives on Supabase's servers; your local machine is not reachable from there.

To test the trigger locally you would need a tunnel such as ngrok or cloudflared, which gives you a temporary public URL. **Or just skip it** — test the endpoint with curl locally, and test the trigger only after deploying. That is simpler, and it is what I would recommend.

## 14.2 In Supabase

- ☐ Run the `login_events` SQL migration from Section 05
- ☐ Verify RLS is enabled and the read-own policy exists
- ☐ Enable the `pg_net` extension
- ☐ Store `login_hook_secret` in Vault
- ☐ Create the trigger function **with its exception handler**
- ☐ Attach the `AFTER INSERT` trigger to `auth.sessions`
- ☐ **Point the URL at your deployed domain, not localhost**

SUPABASE + NEXT.JS + RESEND

### 14.3 In Resend

- ☐ Confirm `fitverse.app` shows **Verified**
- ☐ Confirm DKIM and SPF are present; add DMARC starting at `p=none`
- ☐ Set up `security@fitverse.app` as a sender

### 14.4 In Vercel (or wherever you host)

- ☐ Add `LOGIN_HOOK_SECRET` — identical to the Vault value
- ☐ Add `ADMIN_ALERT_EMAIL`
- ☐ Confirm all six existing variables are present in the Production environment
- ☐ Deploy

### 14.5 Verify in production

- ☐ Log in with a real test account → email arrives
- ☐ Row exists with `email_status = 'sent'`
- ☐ Log in again from another device → second email, second row
- ☐ Leave the app open past token expiry → **no email**
- ☐ Call the endpoint with no secret → **401**
- ☐ Check the email does not land in spam — test Gmail, Outlook, and a phone

#### TIP — AN UNRELATED NOTE ABOUT YOUR BUILD CONFIG

Your production build currently uses `next.config.mjs`, not `next.config.ts`, so the Content Security Policy written in the `.ts` file is not active.

**This does not affect this feature** — CSP governs browsers, and your Resend call is server-side. But if you later fix that config, make sure `connect-src` still permits your Supabase domain, or the app's own login calls will start failing.

## Section 15

## Troubleshooting

When something does not work, find the symptom below and work through the checks in order. Most problems are one of: a mismatched secret, a missing environment variable, or an unverified domain.

### Problem 1 — The user never receives the email

| Possible cause                | What to check                                              | How to fix it                                                                                            |
|-------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| The trigger never fired       | Is there a row in <code>login_events</code> at all?        | No row → the problem is in Supabase (trigger or <code>pg_net</code> ), not in your route. See Problem 4. |
| The route rejected the call   | Your hosting logs — look for 401 responses                 | 401 → the secret in Vault does not match the one in your environment. Re-copy both.                      |
| Resend failed                 | The <code>email_error</code> column on that row            | Read the message — it usually names the exact problem (unverified domain, bad key, invalid address).     |
| The user has no email address | Is <code>email_status</code> set to <code>skipped</code> ? | Working as designed. Check why the account has no address.                                               |
| It went to spam               | The recipient's spam folder                                | See Problem 2.                                                                                           |

### Problem 2 — The email arrives, but goes to spam

| Possible cause            | What to check                                 | How to fix it                                                                             |
|---------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------|
| Domain not fully verified | Resend Dashboard → Domains                    | Complete every DNS record Resend asks for. DNS changes can take a few hours to propagate. |
| Missing DMARC record      | Look up <code>_dmarc.fitverse.app</code>      | Add the DMARC TXT record starting at <code>p=none</code> .                                |
| Spammy subject line       | Is it ALL CAPS, or full of exclamation marks? | Use the recommended subject from Section 8.1.                                             |
| Brand-new sending address | How long has <code>security@</code> existed?  | New addresses have no reputation. It improves as real mail is sent and not marked spam.   |
| No plain-text version     | Are you sending HTML only?                    | Always send both — filters penalise HTML-only mail.                                       |



### Problem 3 — Resend returns an API error

| Possible cause                    | What to check                               | How to fix it                                                                                                                                                   |
|-----------------------------------|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API key missing in production     | Your hosting environment variables          | Add <code>RESEND_API_KEY</code> . <b>Remember your code falls back to "re_placeholder" and fails silently</b> , so a missing key looks like "nothing happened". |
| Sending from an unverified domain | The <code>from:</code> address in your code | It must be on a domain verified in Resend.                                                                                                                      |
| Rate limited                      | Are you retrying in a loop?                 | Cap retries at three attempts (Section 11.2).                                                                                                                   |
| Invalid recipient address         | The address on the user's account           | Mark the row <code>failed</code> and move on — do not retry a malformed address.                                                                                |

### Problem 4 — The webhook/trigger never fires

| Possible cause                                 | What to check                                                | How to fix it                                                                       |
|------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>pg_net</code> not enabled                | Database → Extensions                                        | Enable <code>pg_net</code> . Without it, <code>net.http_post</code> does not exist. |
| Trigger not attached                           | Query <code>pg_trigger</code> for <code>auth.sessions</code> | Re-run your <code>CREATE TRIGGER</code> statement.                                  |
| URL points at localhost                        | The URL inside the trigger function                          | Supabase cannot reach your machine. Use the deployed URL.                           |
| The exception handler is swallowing a real bug | Temporarily log inside the function before the handler       | The handler is protecting your users — do not remove it. Log around it instead.     |
| No session is being created                    | Query <code>auth.sessions</code> directly after logging in   | No new row → the issue is in the app's login code, not this feature.                |

### Problem 5 — Duplicate emails are being sent

| Possible cause                        | What to check                                                                   | How to fix it                                                           |
|---------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| The unique constraint is missing      | Inspect the table definition for <code>unique</code> on <code>session_id</code> | Add it. Without it there is no duplicate protection at all.             |
| You are sending before inserting      | The order of operations in your route                                           | Insert first, then send. Section 6.3, steps 5–9.                        |
| The <code>23505</code> check is wrong | Are you checking <code>error.code</code> ?                                      | Copy the exact check from waitlist route lines 127–131.                 |
| They are genuinely separate logins    | Do the rows have different <code>session_id</code> values?                      | Different IDs = different logins. This is correct behaviour, not a bug. |

## Problem 6 — Login works but no email, and no errors anywhere

| Possible cause                                | What to check                                                 | How to fix it                                                                                                                |
|-----------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Silent Resend failure                         | Is <code>RESEND_API_KEY</code> actually set in production?    | The <code>"re_placeholder"</code> fallback means no crash and no email. This is the most common cause of this exact symptom. |
| Everything is being swallowed                 | Does a row exist, and what is its <code>email_status</code> ? | The row tells you how far the process got. Start there, always.                                                              |
| The trigger's exception handler hid a failure | Whether any row was created at all                            | No row → the failure is upstream, in Supabase.                                                                               |

### TIP — THE UNIVERSAL FIRST QUESTION

Whenever something is not working, ask: "Is there a row in `login_events`, and what does its `email_status` say?"

- **No row** → the problem is in Supabase (trigger, `pg_net`, or the secret).
- **Row says `pending`** → your route crashed midway.
- **Row says `failed`** → read `email_error`; it usually names the cause.
- **Row says `sent`** → the problem is deliverability, not code. Go to Problem 2.

## Problem 7 — Database permission or RLS errors

| Possible cause                         | What to check                                                     | How to fix it                                                                             |
|----------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Using the anon client instead of admin | Are you calling <code>getSupabaseAdmin()</code> ?                 | The anon client obeys RLS and cannot write. Use the admin client in the API route.        |
| RLS enabled with no policy             | Does the read-own policy exist?                                   | With RLS on and no policy, users see nothing. Add the policy from Section 05.             |
| User sees an empty list, expected data | Does <code>auth.uid()</code> match <code>user_id</code> ?         | RLS returns empty results rather than errors — that is by design, so check the IDs match. |
| Foreign key violation on insert        | Does that <code>user_id</code> exist in <code>auth.users</code> ? | Deleted users cannot have new login events. Handle it gracefully.                         |

## Problem 8 — Missing environment variable or invalid secret

| Possible cause            | What to check                                                             | How to fix it                                                  |
|---------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------|
| Every request returns 401 | Compare the Vault value against the hosting value, character by character | A trailing space or newline when pasting is the usual culprit. |

| Possible cause                      | What to check                               | How to fix it                                                                             |
|-------------------------------------|---------------------------------------------|-------------------------------------------------------------------------------------------|
| Variable added but not applied      | Have you redeployed since adding it?        | Most hosts only pick up new environment variables on the next deployment.                 |
| Added to Preview but not Production | Which environment the variable is scoped to | Add it to Production explicitly.                                                          |
| <code>IP_HASH_SALT</code> missing   | Is it set in production?                    | It silently falls back to a value visible in your source code, which weakens the hashing. |

## Section 16

# Final Implementation Checklist

Follow this in order, top to bottom. Steps 1–31 give you working login notifications. Steps 32–35 make it production-grade.

## PHASE 1 • GROUNDWORK

- ☐ 01 Read Sections 1–3 of this guide and understand the architecture
- ☐ 02 Add `.env` to `.gitignore`
- ☐ 03 Confirm `IP_HASH_SALT` is set to a real value in production
- ☐ 04 Read `app/api/waitlist/route.ts` start to finish
- ☐ 05 Submit the waitlist form — confirm Resend works today

## PHASE 2 • SUPABASE

- ☐ 06 Check Authentication → Hooks; note whether Password Verification exists
- ☐ 07 Run the `information_schema` query; note your `auth.sessions` columns
- ☐ 08 Run the `login_events` SQL migration
- ☐ 09 Verify RLS is enabled and the read-own policy exists
- ☐ 10 Enable the `pg_net` extension
- ☐ 11 Generate the shared secret and store it in Vault

## PHASE 3 • RESEND

- ☐ 12 Confirm `fitverse.app` shows Verified
- ☐ 13 Confirm DKIM and SPF; add DMARC starting at `p=none`
- ☐ 14 Set up `security@fitverse.app` as a sender

## PHASE 4 • CODE

- ☐ 15 Add `LOGIN_HOOK_SECRET` and `ADMIN_ALERT_EMAIL` to `.env.local`
- ☐ 16 Write the login email template (HTML + plain text)
- ☐ 17 Build `app/api/auth/login-notify/route.ts`, following Section 6.3
- ☐ 18 Test it with curl — every row of the Stage 2 table
- ☐ 19 Confirm the duplicate test gives exactly 1 row and 1 email

## PHASE 5 • DEPLOY

- ☐ 20 Add both new environment variables to Production
- ☐ 21 Deploy

- ☐ 22 Verify the endpoint returns 401 without a secret

#### PHASE 6 • CONNECT THE TRIGGER

- ☐ 23 Write the trigger function — **with the exception handler**
- ☐ 24 Attach `AFTER INSERT` on `auth.sessions`
- ☐ 25 Point it at your deployed URL

#### PHASE 7 • VERIFY

- ☐ 26 Real login → email arrives, row says `sent`
- ☐ 27 Wrong password → nothing at all happens
- ☐ 28 App restart → no email
- ☐ 29 Wait past token expiry → **NO email** ← the important one
- ☐ 30 Logout → login → second email arrives
- ☐ 31 Break Resend, log in → **login still works** ← the critical one
- ☐ 32 Check inbox placement on Gmail, Outlook and mobile

#### PHASE 8 • PRODUCTION-GRADE

- ☐ 33 Update `app/privacy/page.tsx` to disclose login metadata
- ☐ 34 Add the 90-day retention cleanup job
- ☐ 35 Build the retry job for `pending` / `failed` rows
- ☐ 36 Build the daily admin digest (Section 09)

## Two things to decide before step 17

Both change what you write, so settle them first:

1. **Where does this endpoint live?** This repository, or the app's own backend? If the app gets a separate backend later, you will be moving it.
2. **Every login, or only new devices?** This guide is designed for every login. If you want new-device-only, you need a device fingerprint column in `login_events` — much easier to add now than to retrofit later.

#### ✓ YOU ARE DONE WHEN

A real user logs into FitVerse, receives a clear security email within seconds, sees exactly one row in `login_events` marked `sent` — and when you deliberately break the email system, **that same user still logs in without noticing anything is wrong.**